

Electron

1. Introduction

Electron is a framework used to build **desktop applications using web technologies** such as:

- HTML
- CSS
- JavaScript

Electron allows developers to convert a **web application into a desktop application** that can run on:

- Windows
- macOS
- Linux

Electron combines two important technologies:

1. Chromium

Chromium

- Used to render the graphical user interface.
- Runs web pages like a browser.

2. Node.js

Node.js

- Provides access to system features.
- Enables file system access, process management, and OS operations.

Because of this combination, Electron applications behave like **native desktop applications while internally running web technologies**.

2. Technology Stack

The application uses the following technologies:

Component	Technology
Desktop Framework	Electron
Frontend	React
Build Tool	Vite
Backend API	Node.js
Monitoring Service	ActivityTracker (.NET)
Controlled Browser	KioskBrowser

3. Electron Application Architecture

An Electron application consists of **three main components**:

1. Main Process
 2. Renderer Process
 3. Preload Script
-

3.1 Main Process

The **Main Process** acts as the backend of the Electron application.

Responsibilities:

- Start the application
- Create application windows
- Manage system resources
- Register IPC handlers
- Launch external processes
- Control application lifecycle

Entry file:

main.js

When Electron starts, it first executes this file.

The main process controls the entire application.

3.2 Renderer Process

The **Renderer Process** displays the user interface.

It runs inside Chromium and behaves like a browser.

Responsibilities:

- Render HTML pages
- Run frontend frameworks such as React
- Handle user interactions

Example UI components include:

Login Page

Dashboard

Employee Portal

3.3 Preload Script

The **Preload Script** acts as a secure bridge between the renderer process and the main process.

File:

preload.js

It exposes limited APIs to the frontend.

Example:

```
contextBridge.exposeInMainWorld("electronAPI", {  
  login: (data) => ipcRenderer.invoke("login-controller", data),  
  checkIn: () => ipcRenderer.invoke("checkin"),  
  checkOut: () => ipcRenderer.invoke("checkout")  
});
```

This allows the frontend to communicate with Electron safely.

4. Electron Security Model

Electron applications use important security settings.

contextIsolation: true

nodeIntegration: false

contextIsolation

- Keeps the frontend environment separate from Node.js.

nodeIntegration

- Prevents direct Node.js access from the renderer process.

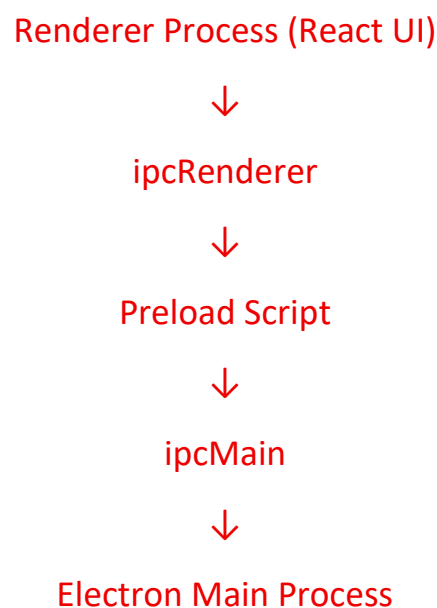
This ensures the application remains secure.

5. How Electron Works

Electron works using **multiple processes**.

Communication between processes occurs through **IPC (Inter Process Communication)**.

Communication flow:



Process explanation:

1. User interacts with UI.
 2. React sends request through ipcRenderer.
 3. Preload script securely forwards the request.
 4. Main process receives the request using ipcMain.
 5. Main process performs system operations.
 6. Response is returned to the UI.
-

6. How Electron Runs the Application

Step 1 — User Starts the Application

The user launches the desktop application.

Electron executes:

`main.js`

This starts the Electron main process.

Step 2 — Electron Initializes Application

The main process prepares the application environment.

Important lifecycle events:

`app.whenReady()`

`window-all-closed`

`before-quit`

`activate`

Responsibilities:

- Register IPC handlers
- Load application modules
- Prepare application window

Step 3 — Create Application Window

Electron creates a desktop window using **BrowserWindow**.

Example:

```
const win = new BrowserWindow({  
  width:1200,  
  height:800,  
  webPreferences:{  
    preload:path.join(__dirname,"preload.js"),  
    contextIsolation:true,  
    nodeIntegration:false  
  }  
});
```

This window loads the frontend application.

Step 4 — Load Frontend Application

Electron loads the frontend inside the window.

Development Mode

`http://localhost:5173`

Frontend runs from the development server.

Production Mode

`frontend/dist/index.html`

Built frontend files are loaded.

Step 5 — Preload Script Connects Frontend and Electron

The preload script exposes Electron APIs to the frontend.

Example APIs:

`window.electronAPI.login()`

`window.electronAPI.checkIn()`

`window.electronAPI.checkOut()`

`window.electronAPI.openBrowser()`

These APIs allow React to communicate with Electron.

Step 6 — User Interaction

Example scenario:

User clicks **Login**.

React calls:

`window.electronAPI.login(credentials)`

This sends an IPC message.

Step 7 — Electron Handles Request

Electron receives the request using IPC handlers.

Controller modules include:

`authController.js`

`breakController.js`

`trackerManager.js`

`kioskBrowser.js`

`windowManager.js`

Example responsibilities:

authController

Handles authentication and attendance.

trackerManager

Starts and stops monitoring services.

kioskBrowser

Launches the restricted browser.

7. Activity Tracker Process

After a successful **Check-In**, Electron launches the monitoring service.

Executable:

ActivityTracker.exe

Arguments passed:

employeeId

attendanceId

The tracker runs silently in the background.

8. Kiosk Browser Launch

Electron can also launch a controlled browser.

Executable:

KioskBrowser.exe

This browser runs as a separate process.

9. Checkout Stops Tracker

When the user clicks **Checkout**:

Electron stops the monitoring service.

Example:

```
trackerManager.stopTracker()
```

10. Application Close

When the application closes:

```
app.on("before-quit")
```

Electron performs cleanup operations:

```
stopTracker()
```

```
stopBrowser()
```

This prevents background services from running after exit.

11. Integrating Frontend with Electron

The renderer process must **not call backend APIs directly**.

Incorrect example:

```
axios.post("http://localhost:5000/api/login")
```

Correct approach:

```
window.electronAPI.login()
```

Electron handles API communication internally.

12. Frontend Integration Steps

Step 1 — Identify API Calls

Search the React project for API calls.

```
grep -rn "axios" src/
```

```
grep -rn "fetch(" src/
```

Step 2 — Expose APIs in preload.js

Example:

```
contextBridge.exposeInMainWorld("electronAPI", {  
  login: (data) => ipcRenderer.invoke("login-controller", data),  
  checkIn: () => ipcRenderer.invoke("checkin"),
```

```
checkout: () => ipcRenderer.invoke("checkout")
});
```

Step 3 — Replace API Calls in React

Before:

```
axios.post("/api/login")
```

After:

```
window.electronAPI.login(data)
```

Step 4 — Create Frontend Service Layer

Example file:

```
src/services/electronService.js
```

Example code:

```
export const electronService = {
  login: (data) => window.electronAPI.login(data),
  checkIn: () => window.electronAPI.checkIn(),
  checkout: () => window.electronAPI.checkOut()
};
```

This centralizes Electron communication.

13. Role-Based Page Integration

The application supports multiple roles.

Roles include:

- Employee
- HR
- Admin

- Team Lead

After login, the backend returns the user role.

Example response:

```
{  
  token:"...",  
  role:"Employee",  
  userId:"123"  
}
```

Frontend redirects users to appropriate dashboards.

Example routing:

Employee → /employee-dashboard

HR → /hr-dashboard

Admin → /admin-dashboard

TeamLead → /teamlead-dashboard

React Router manages navigation.

14. Development Workflow

Run frontend and Electron separately.

Start Frontend

```
cd frontend
```

```
npm run dev
```

Start Electron

```
cd electron
```

```
npm start
```

Electron loads the frontend from the development server.

15. Production Build

Build frontend:

`npm run build`

Output:

frontend/dist

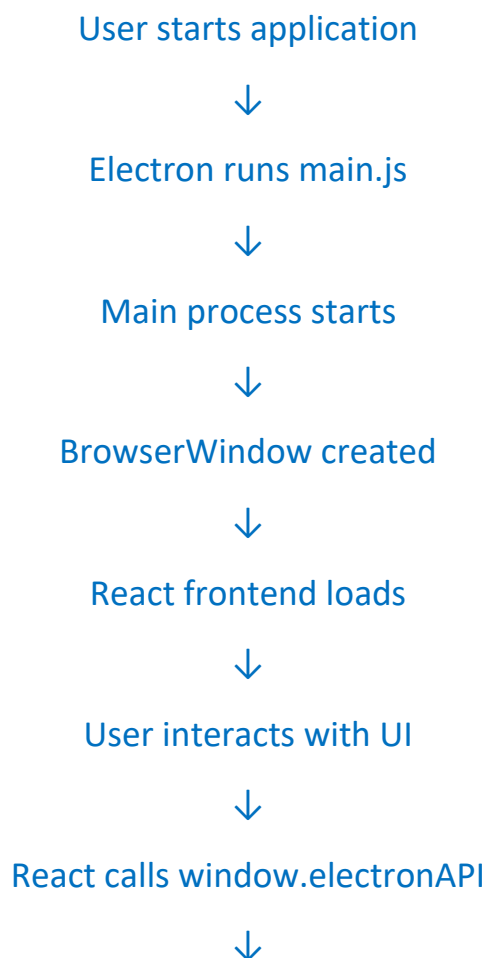
Package Electron application:

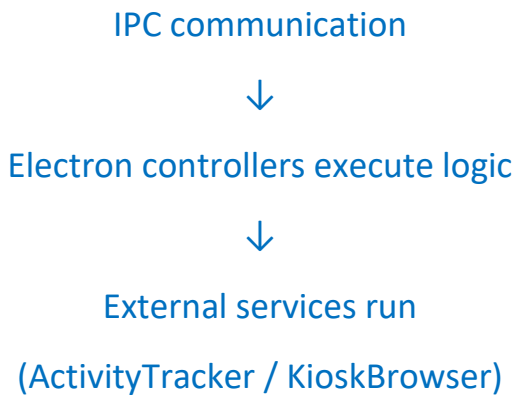
`npx electron-builder --win`

Output:

dist/app-installer.exe

16. Complete Execution Flow





17. Troubleshooting

Electron API not available

Problem:

window.electronAPI is undefined

Solution:

Check preload configuration:

preload: path.join(__dirname, "preload.js")

ActivityTracker not starting

Check:

- ActivityTracker.exe exists
 - correct file path
 - trackerManager.startTracker() called
-

KioskBrowser not launching

Check:

tracker/KioskBrowser.exe
exists in the tracker folder.